

UNIT 4 – TREES (Data Structures)

Trees: Basic terminology used with Tree, Binary Trees, Binary Tree Representation: Array Representation and Pointer(Linked List) Representation, Binary Search Tree, Strictly Binary Tree ,Complete Binary Tree .

A Extended Binary Trees, Tree Traversal algorithms: Inorder, Preorder and Postorder, Constructing Binary Tree from given Tree Traversal, Operation of Insertion , Deletion, Searching & Modification of data in Binary Search . Threaded Binary trees, Traversing Threaded Binary trees. Huffman coding using Binary Tree. Concept & Basic Operations for AVL Tree , B Tree & Binary Heaps

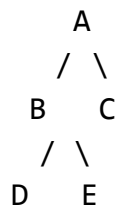
◇ 1. Introduction to Trees

◇ Definition:

A **tree** is a **non-linear data structure** that represents data in a **hierarchical form** (like a family tree).

It consists of **nodes** connected by **edges**.

Example:



Here,

- A → Root node
- B, C → Children of A
- D, E → Leaf nodes

◇ 2. Basic Terminology

Term	Meaning
Node	Each element in a tree.
Edge	Link between parent and child nodes.
Root	Topmost node (has no parent).
Parent	Node with one or more children.
Child	Node derived from parent.
Leaf (Terminal Node)	Node with no children.

Sibling	Nodes having the same parent.
Level	Distance (in edges) from root.
Height of Tree	Maximum level of any node.
Degree of Node	Number of children of a node.
Subtree	Tree formed by a node and its descendants.

◇ 3. Types of Trees

Type	Description
General Tree	Each node can have any number of children.
Binary Tree	Each node has at most two children (Left and Right).
Strictly Binary Tree	Every non-leaf node has exactly two children .
Complete Binary Tree	All levels except possibly the last are full; nodes at the last level are as far left as possible.
Extended Binary Tree	Also called 2n+1 tree — leaf nodes replaced by external nodes (NULL pointers).

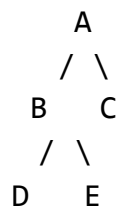
◇ 4. Binary Tree Representation

(a) Array Representation

- The tree is stored level by level in an array.
- For node at index i :
 - **Left child:** $2*i + 1$
 - **Right child:** $2*i + 2$
 - **Parent:** $(i - 1)/2$

Example:

Tree:



Array Representation: [A, B, C, D, E]

Node	Index	Left Child	Right Child
A	0	1	2
B	1	3	4
C	2	-	-

(b) Linked List Representation

Each node has three parts:

```
struct node {  
    data;  
    left pointer;  
    right pointer;  
};
```

Example:

```
struct node {  
    int data;  
    struct node *left;  
    struct node *right;  
};
```

- Root → points to first node
- left → left child, right → right child

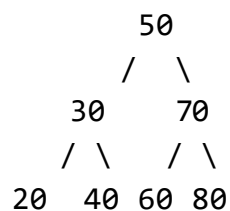
◇ 5. Binary Search Tree (BST)

Definition:

A **BST** is a binary tree where:

- Left child < Parent
- Right child > Parent
- No duplicate values

Example:



(a) Searching in BST

```
if root == NULL → Not found  
if key == root.data → Found  
if key < root.data → search left  
else → search right
```

Time Complexity:

Case	Time
------	------

Best	$O(\log n)$
Worst	$O(n)$ (if skewed tree)

(b) Insertion in BST

1. Compare the new key with root.
2. If smaller $\rightarrow$ go left, if larger $\rightarrow$ go right.
3. Insert when you reach NULL.

(c) Deletion in BST

Cases:

1. **Leaf node** – delete directly.
2. **One child** – replace with its child.
3. **Two children** – replace with inorder successor or predecessor.

(d) Modification in BST

Modify = delete the old value and insert the new value.

◇ 6. Tree Traversals (Visiting all nodes)

There are **3 main types** of depth-first traversals:

Traversal	Order	Example (for tree below)
Inorder	Left $\rightarrow$ Root $\rightarrow$ Right	D B E A F C
Preorder	Root $\rightarrow$ Left $\rightarrow$ Right	A B D E C F
Postorder	Left $\rightarrow$ Right $\rightarrow$ Root	D E B F C A

Example Tree:



◇ Constructing Binary Tree from Traversals

If you are given **Inorder + Preorder** (or **Inorder + Postorder**), you can **rebuild** the tree.

Rule:

- In preorder $\rightarrow$ first element = root.
- Use inorder to divide into left and right subtrees.

◇ 7. Threaded Binary Tree

Problem with Normal Binary Tree:

Many NULL pointers waste space.

Idea:

Use these NULL pointers to store the address of **inorder predecessor or successor**.

Advantages:

- Fast traversal without recursion or stack.
- Saves memory.

(a) Types

- **Single Threaded:** Each node points to either successor or predecessor.
- **Double Threaded:** Each node points to both predecessor and successor.

Traversal:

Start from leftmost node → follow right threads until NULL.

◇ 8. Huffman Coding using Binary Tree

Used for **data compression**.

Idea:

Characters with **higher frequency** → **shorter code**

Characters with **lower frequency** → **longer code**

Steps:

1. Create leaf nodes for each character and its frequency.
2. Combine two smallest frequency nodes into one.
3. Repeat until one root remains.
4. Assign 0 for left edge, 1 for right edge.

Example:

Character	Frequency	Huffman Code
A	5	10
B	9	11
C	12	0

Used in: ZIP, JPEG, MP3, etc.

◇ 9. AVL Tree (Height-Balanced BST)

Definition:

An AVL tree is a **self-balancing binary search tree**.

Condition:

For every node,

$$|\text{Height}(\text{left}) - \text{Height}(\text{right})| \leq 1$$

(a) Balance Factor

Balance Factor = height(left subtree) - height(right subtree)

(b) Rotations Used for Balancing

Type	When used
LL Rotation	Left heavy after insertion in left subtree
RR Rotation	Right heavy after insertion in right subtree
LR Rotation	Left-Right heavy
RL Rotation	Right-Left heavy

Advantages:

- Fast searching ($O(\log n)$)
- Always balanced

Disadvantage:

- More complex insertion and deletion.

◇ 10. B-Tree (used in Databases)

Definition:

A **B-tree** is a **self-balancing search tree** where each node can have multiple keys and children.

Properties:

- All leaves at same level.
- Each node can store **more than one key**.
- Used for **disk-based searching (database indexing)**.

Order (m):

- Each node can have max **m children**.
- Each node has **m-1 keys**.

Operations: Searching, Insertion, Deletion (done in $O(\log n)$).

◇ 11. Binary Heap

Definition:

A binary heap is a **complete binary tree** that satisfies the **heap property**:

- **Max Heap:** Parent $\geq$ Children
- **Min Heap:** Parent $\leq$ Children

Representation:

Usually implemented using **arrays**.

Operations:

Operation	Time Complexity
Insertion	$O(\log n)$
Deletion (Root)	$O(\log n)$
Find Max/Min	$O(1)$

Applications:

- Priority Queue
- Heap Sort
- Graph Algorithms (Dijkstra, Prim)

◇ 12. Time Complexity Summary

Operation	Binary Tree	BST	AVL Tree	B-Tree	Heap
Insertion	$O(n)$	$O(\log n)$ avg	$O(\log n)$	$O(\log n)$	$O(\log n)$
Deletion	$O(n)$	$O(\log n)$ avg	$O(\log n)$	$O(\log n)$	$O(\log n)$
Searching	$O(n)$	$O(\log n)$ avg	$O(\log n)$	$O(\log n)$	$O(\log n)$

Summary

- **Tree** = hierarchical data structure.
- **Binary Tree** = max two children.
- **BST** = left < root < right.
- **Traversal** = visiting nodes (Inorder, Preorder, Postorder).
- **Threaded Tree** = use NULLs for fast traversal.
- **Huffman Tree** = used in compression.
- **AVL Tree** = height-balanced BST.
- **B-Tree** = used in databases.
- **Heap** = used in priority queues and heap sort.

Important Difference Tables (Unit 4: Trees)

1. Tree vs Binary Tree

Feature	Tree	Binary Tree
Definition	A non-linear data structure with nodes connected in a hierarchy. Each node can have <i>any number</i> of children.	A special tree where each node can have <i>at most two children</i> — left and right.
Children per Node	Any number	Maximum 2
Representation	General tree representation	Linked list or array representation
Traversal Methods	Usually level-order or general recursive	Preorder, Inorder, Postorder
Example	Folder structure in a computer	Binary Search Tree (BST)

2. Strictly Binary Tree vs Complete Binary Tree

Feature	Strictly Binary Tree	Complete Binary Tree
Definition	Every non-leaf node has <i>exactly two children</i> .	All levels except possibly the last are <i>completely filled</i> , and all nodes are as <i>left as possible</i> .
Structure	Each node has 0 or 2 children	Last level may not be full
Example	Perfect Binary Tree	Binary Heap
Usage	Theoretical structure for recursion problems	Used in Heap implementations
Full Levels Required	Not necessarily	Yes, except the last level

3. Array Representation vs Linked List Representation of Binary Tree

Feature	Array Representation	Linked List Representation
Storage Method	Tree nodes are stored in array indices based on their position.	Each node stores data and pointers to left & right child.
Space Utilization	May waste space (especially in unbalanced trees).	Efficient for sparse trees.
Access Time	Fast using indices.	Slower as you traverse via pointers.
Implementation Complexity	Simple for Complete Binary Trees.	Preferred for dynamic and irregular trees.
Example	Binary Heap	Binary Search Tree

4. Binary Search Tree (BST) vs Binary Tree

Feature	Binary Tree	Binary Search Tree (BST)
---------	-------------	--------------------------

Structure Rule	No specific order among nodes.	Left child < Root < Right child.
Searching Efficiency	O(n) (may need to check all nodes).	O(log n) for balanced BST.
Insertion / Deletion	No specific rule.	Must maintain order property.
Example	Expression Tree	Tree storing sorted student roll numbers
Use Case	General tree problems	Searching and sorting data efficiently

5. Binary Tree Traversals

Traversal Type	Order of Visit	Example (for tree root = A)
Inorder	Left → Root → Right	B A C
Preorder	Root → Left → Right	A B C
Postorder	Left → Right → Root	B C A
Level-order	Level by level (top to bottom)	A B C

6. Threaded Binary Tree vs Normal Binary Tree

Feature	Normal Binary Tree	Threaded Binary Tree
Null Pointers	Left or right pointer can be NULL.	Null pointers replaced by <i>threads</i> (links to inorder predecessor/successor).
Traversal	Needs recursion or stack.	Traversal can be done without recursion/stack.
Speed	Slower for large trees.	Faster inorder traversal.
Memory Use	Some wasted space (null links).	Efficient use of pointers.

7. AVL Tree vs Binary Search Tree (BST)

Feature	BST	AVL Tree
Balance Condition	Not necessarily balanced.	Balanced BST: height difference ≤ 1 for every node.
Search Time (Average)	O(log n) if balanced, else O(n).	Always O(log n).
Rotations	No rotations used.	Uses rotations (LL, RR, LR, RL).
Implementation	Simple	More complex due to balancing
Usage	Simple data storage	Efficient searching applications

8. B Tree vs Binary Search Tree

Feature	Binary Search Tree	B Tree
Children per Node	Max 2	Multiple (M-way tree)
Use Case	In-memory operations	Database and Disk storage
Height	More height (deep)	Lesser height (shallow)
Balance	May be unbalanced	Always balanced

Storage Medium	RAM	Disk / External Storage
-----------------------	-----	-------------------------

9. AVL Tree vs B Tree

Feature	AVL Tree	B Tree
Structure Type	Binary (2 children max)	M-way (multiple children)
Used For	Memory (RAM) searching	Disk-based searching
Balancing Method	Height-balanced using rotations	Node-based balance using splits/merges
Access Time	Very fast for small data	Efficient for large database records
Applications	Compiler symbol tables	File systems, Databases

10. Binary Heap vs Binary Search Tree

Feature	Binary Heap	Binary Search Tree
Property	Complete binary tree with heap property (parent > children for max heap).	Follows order property (left < root < right).
Traversal Order	No fixed order.	Inorder gives sorted data.
Usage	Priority queues, heap sort	Searching & dynamic data storage
Time for Search	$O(n)$	$O(\log n)$
Time for Insert/Delete	$O(\log n)$	$O(\log n)$

Notes by -[GKM](#)